

```

PreparedStatement ps = conn.prepareStatement(updateSQL);
while (rs.next()) {
    /* 阅读每一条课程评价 */
    String strSno = rs.getString("Sno");
    String strAssess = rs.getString("Assess");
    Boolean bCAtype = rs.getString("CAtype");
    String strFeedback = rs.getString("Feedback");
    /* 打印课程评价详情,注意隐藏学生的信息,例如学生的学号 */
    System.out.printf("[%s,%b,%s]%n",strAssess, bCAtype, strFeedback);
    /* 对该评价进行回复 */
    ps.setString(1, "感谢你的评价");          /* 反馈意见根据教师的输入进行设置 */
    ps.setString(2, strSno);                    /* 设置学生学号 */
    ps.setString(3, strTno);                    /* 设置教师职工号 */
    ps.setString(4, strTCno);                  /* 设置教学班号 */
    ps.executeUpdate();                        /* 更新教师课程意见反馈 */
}
rs.close();
ps.close();
stmt.close();
ConnectionManager.release();

```

从例 8.13 可以看出,在应用程序中直接使用 JDBC 操纵数据库,需要经过图 8.6 所示的加载驱动程序—建立与数据库的连接—执行 SQL 语句—处理结果集—释放资源的整个过程(加载驱动程序可以省略)。程序的代码管理显得比较复杂。此外,在程序的模块中需要获取数据库连接,在数据操作完毕之后再关闭数据库连接。这样对数据库进行频繁连接、开启和关闭操作,会造成数据库资源的浪费,影响数据库的性能。8.4 节基于 MVC 框架的数据库应用开发可以克服这些缺点。

* 8.4 基于 MVC 框架的数据库应用开发

MVC(model-view-controller, 模型-视图-控制器模式)是一种使用业务模型(Model)、用户结果呈现(View)以及业务模型控制器(Controller)来进行 Web 数据库应用开发的编程框架。基于 MVC 框架进行 Web 数据库应用开发,其核心思想在于功能的解耦以及功能的模块化。以本书第 7 章中介绍的“高校本科教务管理”信息系统的子系统“教师教学管理”为例进行介绍(参见图 7.30),该子系统共包括“教师”“教室”“时间片”“教学班”等实体,以及“排课”“讲授”“课程评价”等联系。其功能包括:

- ① “教师”实体的增加、删除、修改、查询。
- ② “教室”实体的增加、删除、修改、查询。
- ③ “时间片”实体的增加、删除、修改、查询。
- ④ “教学班”“教室”“时间片”三者之间“排课”联系的增加、删除、修改、查询。
- ⑤ “教学班”“教师”之间“讲授”联系的增加、删除、修改、查询。
- ⑥ “学生”“教学班”“教师”三者之间“课程评价”与回复的增加、删除、修改、查询。

基于 MVC 框架进行应用开发,需要将上述 6 个功能模块进行解耦,并按照 MVC 框架的工作流程图对每一个模块进行独立设计与实现。

图 8.8 给出了 MVC 框架的工作流程图。MVC 框架包含三部分:业务模型控制器、业务模型和用户结果呈现。每部分各自处理自己的工作,互不干扰。

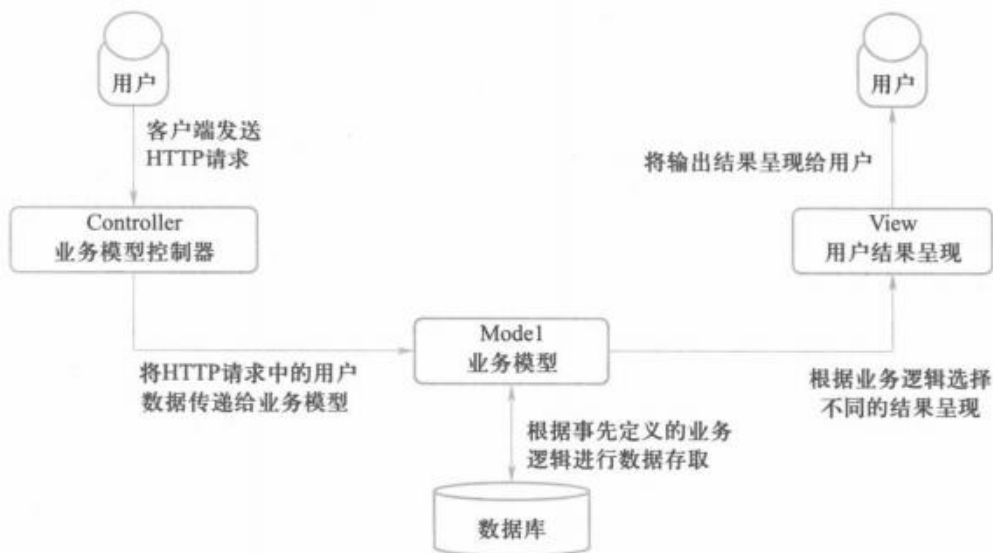


图 8.8 MVC 框架工作流程图

业务模型控制器用来接收用户通过浏览器发送的 HTTP 服务请求,该请求中包括用户需要的业务模型,以及其他可选的输入参数(可以是用户在文本框中输入的文本、下拉列表框中的数据项等,也可以是用户的操作类型)。

业务模型用来建立用户的业务逻辑。它根据业务模型控制器解析出来的用户数据,执行事先定义的业务逻辑,并操纵数据库存取数据。业务模型的输出独立于具体的数据格式(考虑到数据源的多样性),可以为多个用户结果呈现提供展示所需要的数据。由于业务模型可以被多个用户结果呈现重用,所以减少了代码的重复性。

用户结果呈现是用户看到并与之交互获得结果展示的界面。它根据业务模型输出的结果反馈给客户端进行呈现。

在 MVC 开发框架中,从应用程序与数据库交互的角度来看,其在应用程序中无须显式地

使用 JDBC 操纵数据库,而仅需要在业务模型中显式地建立业务与数据库之间的映射关系,由 MVC 框架自动完成应用程序与数据库之间的交互(具体可参见例 8.14)。

[例 8.14]使用 MVC 框架实现任务 4。

图 8.9 给出了使用 MVC 框架实现任务 4 的系统流程图,ClassAssessController 类对应业务模型控制器,ClassAssessModel 类对应业务模型,ClassAssessView 类对应用户结果呈现。

ClassAssessController 类分别创建了 ClassAssessService 类对象和 ClassAssessModel 类对象。ClassAssessService 类对象用来接收用户通过浏览器提交的 HTTP 请求,解析请求中用户的输入参数。以教师用户访问“教师浏览指定教学班评价与回复”功能页面为例(如图 8.9 左上角显示,用户可以通过“高校本科教务管理”信息系统主页导航到该功能页面),当教师用户浏览到学生对课程的评价并需要做出意见回复时,可以在相应的文本框中输入对学生课程评价的意见反馈,点击“回复”按钮进行意见反馈提交。Web 服务器通过解析用户的提交请求(例如图中给出的链接请求),将该 Web 请求发送给指定的业务模型控制器 ClassAssessController 类。需要注意的是,该 Web 请求中还包括请求的用户输入(即传递的变量):职工号、教学班号、学生学号、教师对学生课程评价的意见反馈。根据用户的操作请求,业务模型控制器将用户输入传递给 ClassAssessModel 类对象中的 insertFeedback 方法,把教师对指定学生课程评价的意见反馈存放到数据库中。

ClassAssessModel 类用来实现学生课程评价以及教师回复学生课程评价的应用逻辑。方法 insertFeedback 以控制器传递的学生学号 Sno、教学班号 TCno、教师职工号 Tno、教师反馈 Feedback 作为参数,将教师回复的学生评价反馈更新到数据库中。方法 getAssess 以控制器传递教学班号 TCno、教师职工号 Tno 为查询参数,查询出数据库中当前教学班所有学生的评价信息。结合方法 insertFeedback 和方法 getAssess,可以实现将教师回复指定学生课程评价的意见反馈数据更新到数据库中,并查询出数据库中当前教学班所有的最新评价数据返回 ClassAssessView 类展示给用户。方法 getTeachingClass 和方法 insertAssess 分别实现学生浏览所选教学班信息以及插入学生的课程评价信息功能。

在 MVC 开发框架中,业务模型与数据库之间的交互主要包括数据访问对象(data access object, DAO)和对象/关系数据库映射(object-relational mapping, ORM)。DAO 是将所有对数据源的访问操作抽象封装在这个类中。在应用程序中,当需要和数据源进行交互时则使用 DAO 类提供的方法进行统一访问。简单地说,DAO 就是将数据库中数据的增加、删除、修改、查询等操作封装在专门的类中,当业务模型需要与数据库进行交互时,直接调用该 DAO 类来实现 JDBC 操纵数据库的每一个步骤。

ORM 是将关系数据库中关系模式和业务模型之间做一个映射(例如关系模式 ClassAssess 和 ClassAssessModel),并建立业务模型中成员变量与关系模式属性之间的映射关系。这样,当与数据库进行交互时无须撰写复杂的 SQL 语句与数据库打交道,而只需要在应用程序中操作业务模型中的成员变量。数据库持久层框架 iBatis、myBatis、Hibernate,以及很多 PHP 框架都自带 ORM 库,在 MVC 框架中引入数据库持久层框架,通过撰写简单的 XML 配置文件即可实现关系模式和业务模型之间的映射,从而极大地简化业务模型与数据库之间的开发复杂度。

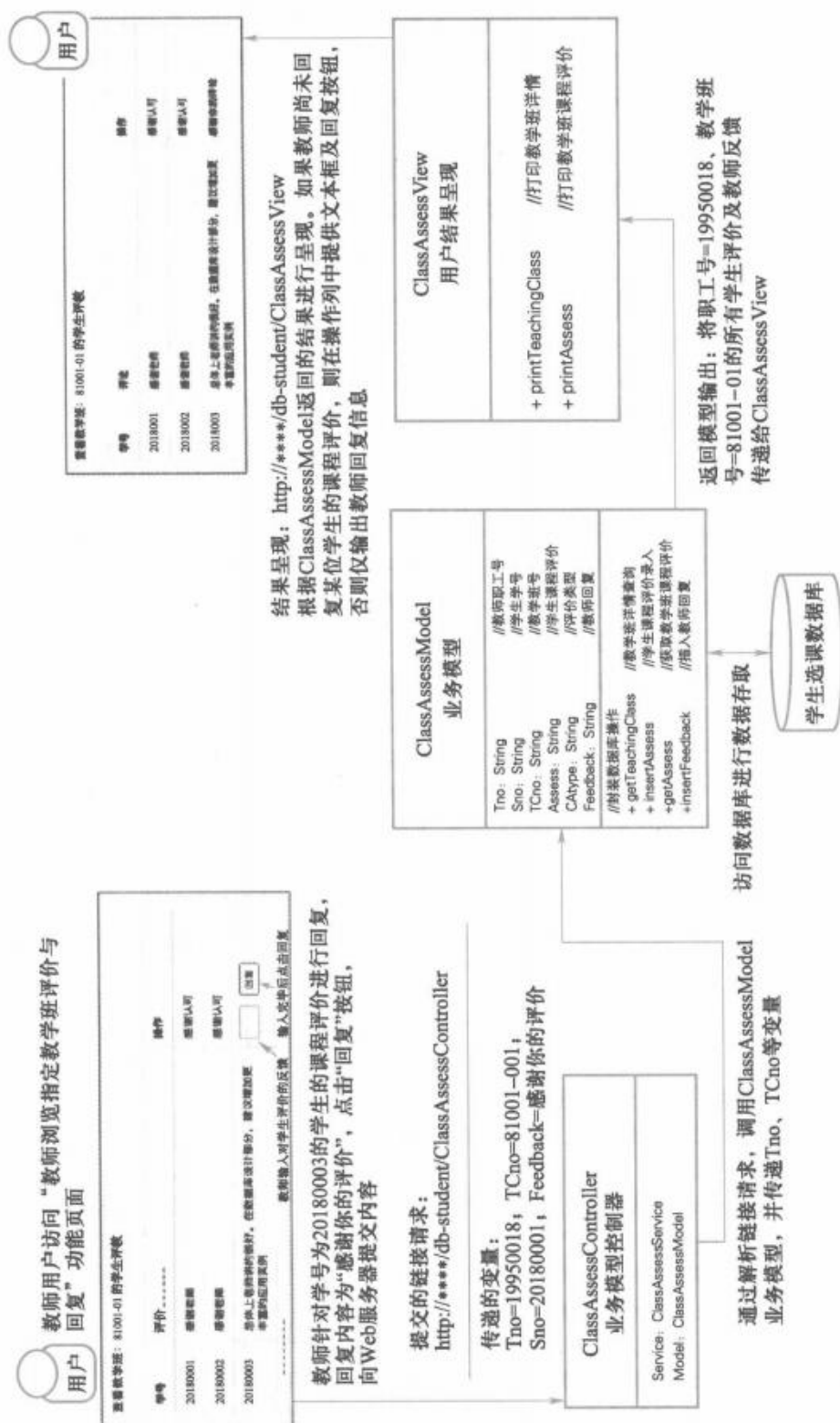


图 8.9 使用 MVC 框架实现任务 4 的系统流程图

ClassAssessView 类用来将业务模型的执行结果按照合适的方式进行结果呈现。例如, 类中的 printTeachingClass 函数实现的功能就是打印学生浏览其所选教学班及其授课教师。

由于 MVC 框架可以使用配置文件的方式支持业务模型与数据库之间的数据交互, 避免了在业务模型中显式地调用 JDBC 编程与数据库进行交互, 从而降低了应用开发的复杂度。另外, MVC 框架提供的连接池管理优化了应用程序频繁访问数据库的性能。进一步地, MVC 框架中的业务模型、用户结果呈现、业务模型控制器分离架构, 使程序更加容易开发和维护。值得一提的是, 在实际的应用开发中, 根据应用的类型、用户规模的大小, 所选用的开发模式也会有所不同。本书限于篇幅, 不再介绍其他开发模式。

本章小结

本章从两个方面讲解数据库编程技术: 一是扩展 SQL 的功能, 二是通过高级语言实现复杂应用。

本章讲解了扩展 SQL 能力的技术途径, 具体包括引入新的 SQL 子句、引入新的内置函数、引入 PL/SQL 以及存储过程/存储函数等技术。存储过程和存储函数经编译和优化后存储在数据库服务器中, 运行效率高, 客户机和服务器之间的通信量小, 可以集中控制管理, 因此被广泛使用。

SQL 与高级语言具有不同的数据处理方式。SQL 是面向集合的, 而高级语言是面向记录的。游标就是用来协调这两种不同的处理方式的机制。要掌握游标的概念, 学会用游标来编写实际的应用程序。

本章还阐述了 JDBC 的工作原理和工作流程, 即在 Java 程序中如何访问数据库中的数据。使用 JDBC 编写的应用程序可移植性好, 能同时访问不同的数据库, 共享多个数据资源。本章最后讲解了基于 MVC 框架的开发方式, 在实际的项目开发中, 采用开发框架的方式可以提高应用开发的效率。

习 题 8

1. 假定一门课程存在多门直接先修课程, 使用 WITH RECURSIVE 子句查找“数据库系统概论”课程的所有先修课课程号和课程名称。
2. 对“学生选课”数据库编写存储过程, 完成下述功能:
 - ① 统计“离散数学”课程的成绩分布情况, 即按照各分数段统计人数。
 - ② 统计任意一门课程的平均成绩。
 - ③ 将学生选课成绩从百分制改为等级制(即 A、B、C、D、E)。
3. 设有产品表的关系模式 Product(PID, PName, Price, Amount, DateOfManufacture), 描述的是产品编号、产品名称、单价、数量、生产日期, 其中主码为产品编号与生产日期。请基于某一种关系数据库管理系统, 使用数据定义语言完成 Product 关系模式的创建, 并使用存储过程或存储函数生成不少于 1 000 条产品记录。
4. 使用 JDBC 编写应用程序, 实现对异构数据库中的数据进行迁移。要求:
 - ① 配置两种不同的数据源, 编写程序连接两种不同关系数据库管理系统的数据源。
 - ② 分别在上述两种关系数据库管理系统中完成第 3 题产品关系模式创建及产品记录的生成。

③ 使用 JDBC 编写应用程序，将一种关系数据库管理系统产品表的数据迁移到另一种关系数据库管理系统的产品表，进行追加。

第 8 章实验 数据库编程与大作业

实验 8.1 存储过程实验。掌握数据库 PL/SQL 编程语言，以及数据库存储过程的设计和使用方法，包括存储过程定义、存储过程运行、存储过程更名、存储过程删除、存储过程的参数传递。掌握 PL/SQL 编程语言和编程规范，规范设计存储过程。

实验 8.2 自定义函数。掌握数据库 PL/SQL 编程语言以及数据库自定义函数的设计和使用方法，包括自定义函数定义、自定义函数运行、自定义函数更名、自定义函数删除、自定义函数的参数传递。掌握 PL/SQL 和编程规范，规范设计自定义函数。

实验 8.3 游标实验。掌握 PL/SQL 游标的设计、定义和使用方法，理解 PL/SQL 游标按行操作和 SQL 按结果集操作的区别和联系。掌握各种类型游标的特点、区别与联系。

实验 8.4 基于 JDBC 访问数据库的应用开发。掌握基于 JDBC 驱动的数据库应用开发方法、基于 JDBC 驱动的数据库连接方法，实现数据库数据操纵等应用开发常见功能。

实验 8.5 数据库设计与应用开发大作业。掌握综合运用数据库原理、方法和技术进行数据库应用系统分析、设计和开发的能力。能够针对某部门或单位的应用需求，通过系统分析，从数据库数据和应用系统功能两方面进行综合设计，实现一个完整的数据库应用系统。

参考文献 8

- [1] RDBMS 用户使用手册。
- [2] RDBMS 编程手册。
- [3] 关于 Java 和 JDBC，可参阅 Oracle 官网上的 Java Tutorials 文档，也可从 MSDN(Microsoft Developer Network)上获得相关的资料。
- [4] 王珊，张俊．数据库系统概论(第 5 版)习题解析与实验指导[M]．北京：高等教育出版社，2014。

本章知识点讲解微视频：



SQL 语言表达能力的限制



过程化 SQL 例子



存储过程的使用场景



JDBC 架构与工作原理

第三篇

系 统 篇

本篇讲解数据库管理系统中查询处理和事务管理的基本概念和基础知识。

本篇包括 5 章。

第 9 章关系数据库存储管理。存储管理是数据库管理系统的重要职责之一，它与查询优化以及数据库物理设计都密切相关。本章介绍关系数据库的数据组织方法和 5 种常用的索引结构。

第 10 章关系查询处理和查询优化，讲解关系数据库管理系统查询处理的基本步骤，实现选择操作和连接操作的算法，查询优化的概念、方法和技术，以及查询计划的执行方法。

第 11 章数据库恢复技术，讲解事务的基本概念和性质，数据库系统遇到故障后进行恢复的技术和方法，以及提高故障恢复效率的检查点技术和数据库镜像技术。

第 12 章并发控制，介绍并发操作可能造成数据不一致的问题，以及事务的 4 种隔离级别；讲解并发控制的基本概念和最常用的封锁技术；讨论并发调度的可串行性，以及如何用封锁技术保证并发事务的可串行性。

* 第 13 章数据库管理系统，阐述数据库管理系统的基本功能、系统结构及主要的实现技术。本章可以作为选读内容，为读者进一步学习数据库内核实现技术打下基础。

第9章

关系数据库存储管理

本章讨论关系数据库的数据组织和存储管理。我们将重点介绍基于磁盘的数据库的组织与存储，有关内存数据库的数据组织见新技术篇的第17章。

本章导读

存储管理是数据库管理系统(DBMS)的重要职责之一。它与查询优化以及数据库物理设计都密切相关。

本章介绍关系数据库的数据组织和索引结构。在9.1节数据组织中主要讨论数据库的逻辑与物理组织方式，重点介绍记录的表示、记录如何在块中组织存储以及如何组织关系表的存放；在9.2节索引结构中重点介绍5类索引，包括顺序表索引、辅助索引、B+树索引、哈希索引和位图索引。

本章的难点是理解数据库逻辑组织与物理组织之间的对应关系，了解关系表的几种存放方式，深入理解索引的作用，掌握B+树索引和哈希索引的组织方式和查找方式。

在第一篇基础篇中我们已经知道，关系数据库的一个重要特点是存储结构和存取路径对用户隐蔽，用户访问数据库时，只需要用SQL语句表达自己需要哪些信息，比如“查询数据库系统概论课程的选课人数”，而不需要像过程化语言那样具体指定怎么去查找。将用户的查询请求转化成查询执行计划，是数据库查询优化的一个重要工作，而了解关系数据库的数据组织与存储管理，能帮助我们更好地理解关系数据库查询优化的原理。同时，了解数据组织与存储管理，也能帮助我们在应用开发中更好地进行数据库物理设计。

9.1 数据组织

数据库是大量数据的有结构的综合性集合。如何将这样一个庞大的数据集合以最优的形式组织起来存放在外存上，是一个非常重要的问题。所谓“优”应包括两方面：一是存储效率高，节省存储空间；二是存取效率高，访问速度快，代价小。本节主要介绍数据库的逻辑组织方式和物理存储方式。